Is RAG Dead?: Anthropic Says No

8 min read · May 28, 2025

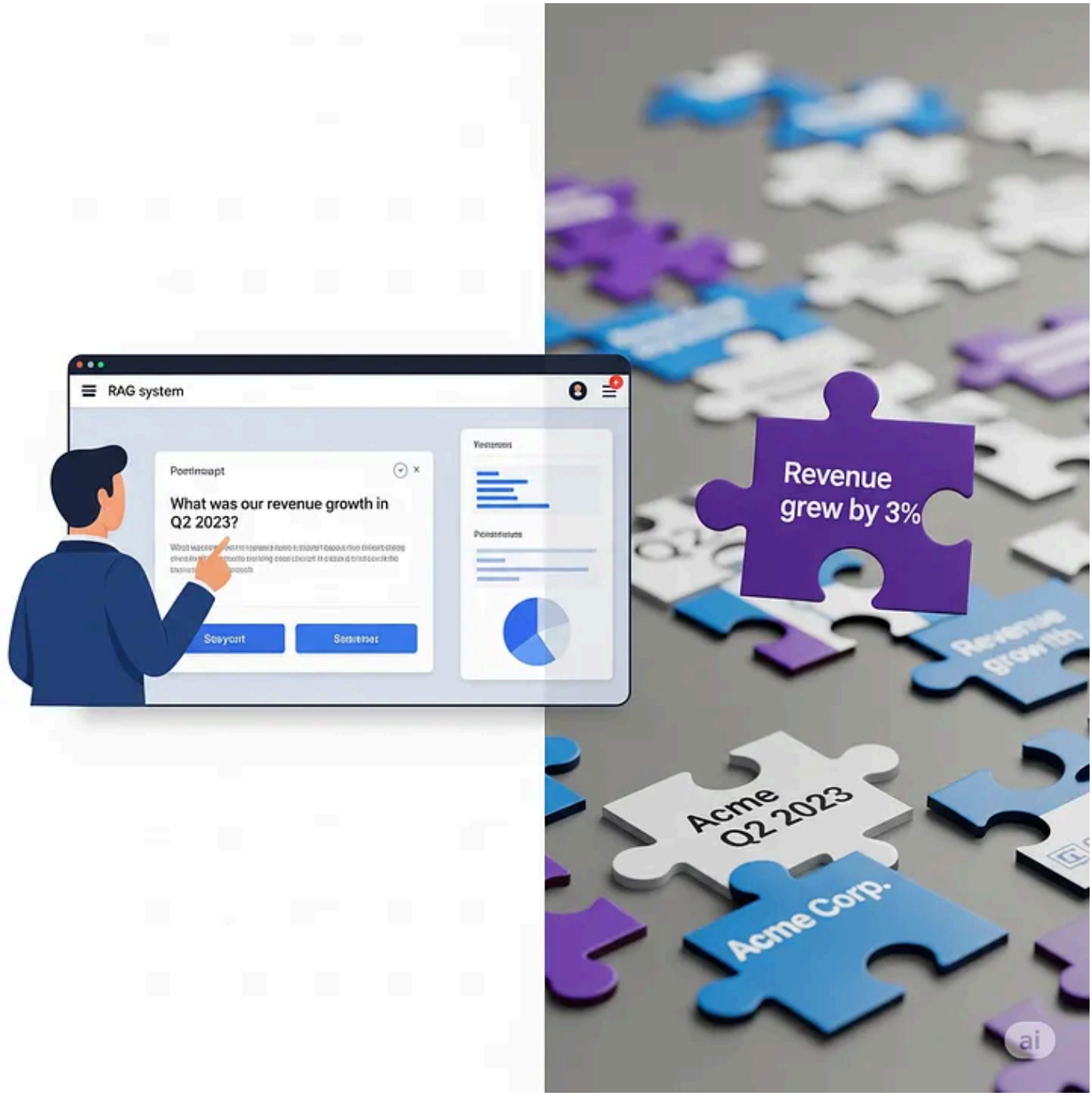

Rick Hightower Follow

Is your RAG system not giving clear answers? Anthropic's new contextual retrieval approach could transform how your system processes and retrieves data. Learn how to enhance accuracy and get smarter responses in this must-read article.

Many developers have struggled with RAG systems' limitations, which is why Anthropic's contextual retrieval approach has generated significant industry interest. Others have said RAG is dead, and you should just use CAG, but what if your knowledge base doesn't fit.

Your RAG System Is Forgetting Where It Put Its Keys: How Anthropic's Contextual Retrieval Solves the Context Problem

Ever asked your RAG system a specific question only to get back a frustratingly vague answer? You're not alone. Here's how a clever enhancement is changing the game.



Picture this: You've built a sleek RAG (Retrieval Augmented Generation) system for your company's documentation. You ask it, "What was our revenue growth in Q2 2023?" and it confidently responds with "Revenue grew by 3% over the previous quarter." Great, except... which company? Which quarter? Your system just handed you a puzzle piece without showing you the box.

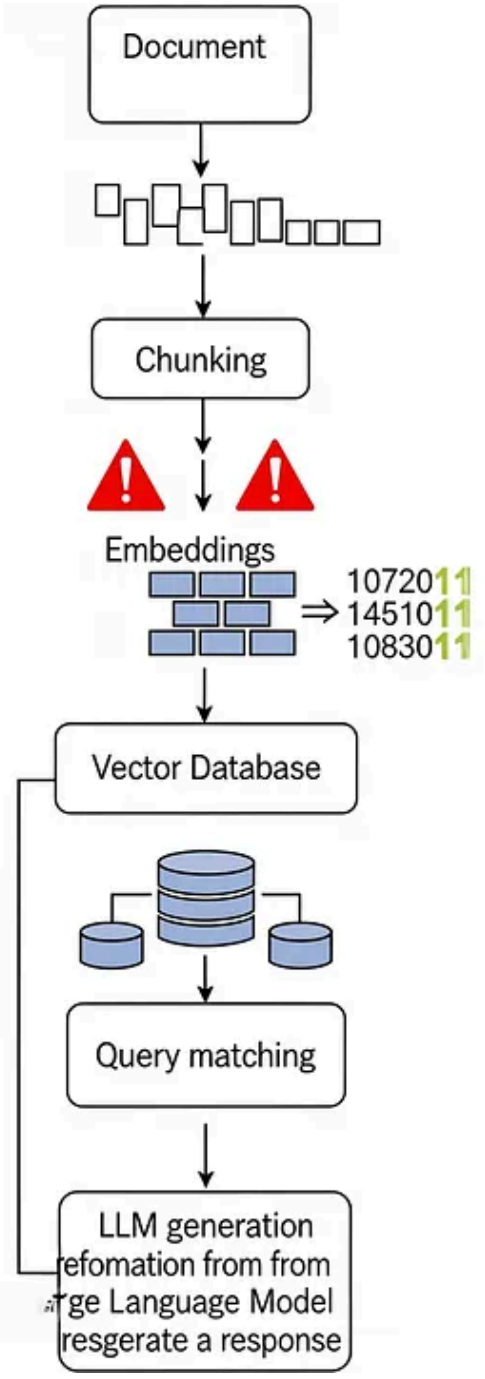
If you've worked with RAG systems, you've probably hit this wall. The good news? Anthropic recently introduced a deceptively simple solution called "contextual retrieval" that's showing remarkable results — up to 49% reduction in retrieval failures when combined with existing techniques.

Let's dive into what makes this approach special and why you might want to implement it in your next project.

The Context Problem: When Good Chunks Go Bad

Before we get into the solution, let's understand why standard RAG systems struggle with context. In a typical RAG pipeline, you're doing something like this:

1. Break documents into chunks (usually a few hundred tokens each)
2. Generate embeddings for each chunk
3. Store these embeddings in a vector database
4. When a query comes in, find the most semantically similar chunks
5. Feed these chunks to your LLM to generate an answer



This works beautifully for many use cases. Ask about “machine learning best practices” and you’ll likely get relevant chunks about ML techniques. But here’s where things get tricky.

Imagine your document contains multiple quarterly reports for different companies. When you chunk the document, that critical piece saying “revenue grew by 3%” gets separated from the context that identifies it as “Acme Corp’s Q2 2023 performance.” Your embedding model sees “revenue growth” and thinks “this is financially relevant!” but has no idea which company or time period it refers to.

The Band-Aid Approach: Keyword Search to the Rescue?

Many developers have tried to patch this problem by adding keyword search (like BM25) alongside semantic search. This helps when you're looking for specific terms like error codes. Search for "Error TS-99" and keyword search will nail it, while pure semantic search might return general troubleshooting guides.

But even this dual approach has limitations. If your documents mention "revenue grew by 3%" multiple times for different contexts, keyword search alone won't know which instance you need. You're still missing that crucial contextual information.

Enter Contextual Retrieval: Teaching Chunks to Remember Where They Came From

Here's where Anthropic's approach gets clever. Instead of trying to fix the retrieval mechanism, they enhance the chunks themselves. The core idea is elegantly simple: before storing a chunk, prepend it with relevant contextual information.

Let's look at our revenue example:

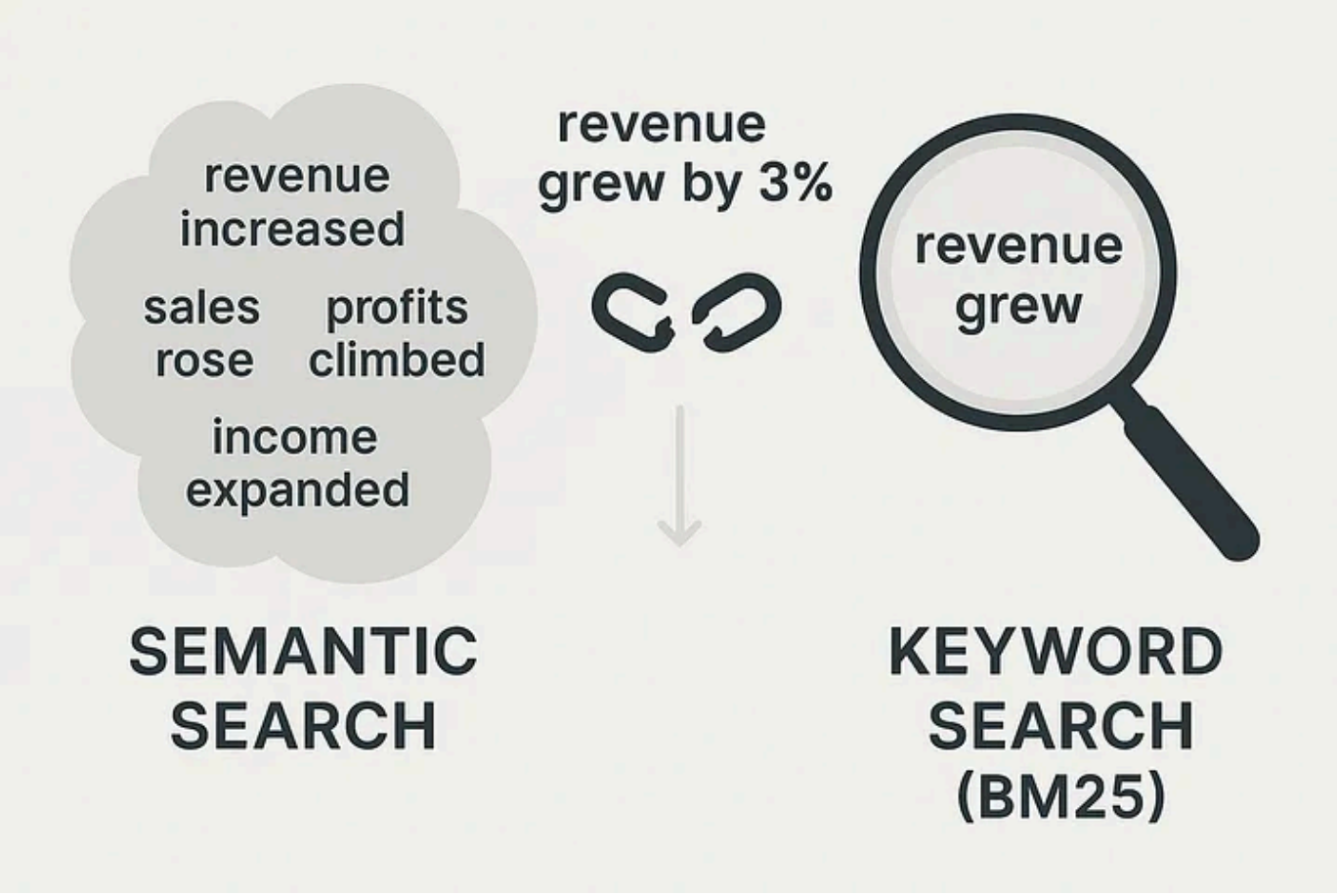
Original chunk: "Revenue grew by 3% over the previous quarter"

Contextualized chunk: "This chunk is from a filing on Acme Corporation's performance in Q2 2023. Revenue grew by 3% over the previous quarter"

Now both your embedding model and keyword search have the full picture. When someone searches for "Acme Q2 2023 revenue," both search mechanisms can confidently identify this as the relevant chunk.

The Implementation: Making LLMs Do the Heavy Lifting

You might be thinking, "Great idea, but I'm not manually adding context to thousands of chunks." This is where the implementation gets interesting. Anthropic suggests using an LLM (they recommend their efficient Claude Haiku model) to automatically generate this context.



Here's the process:

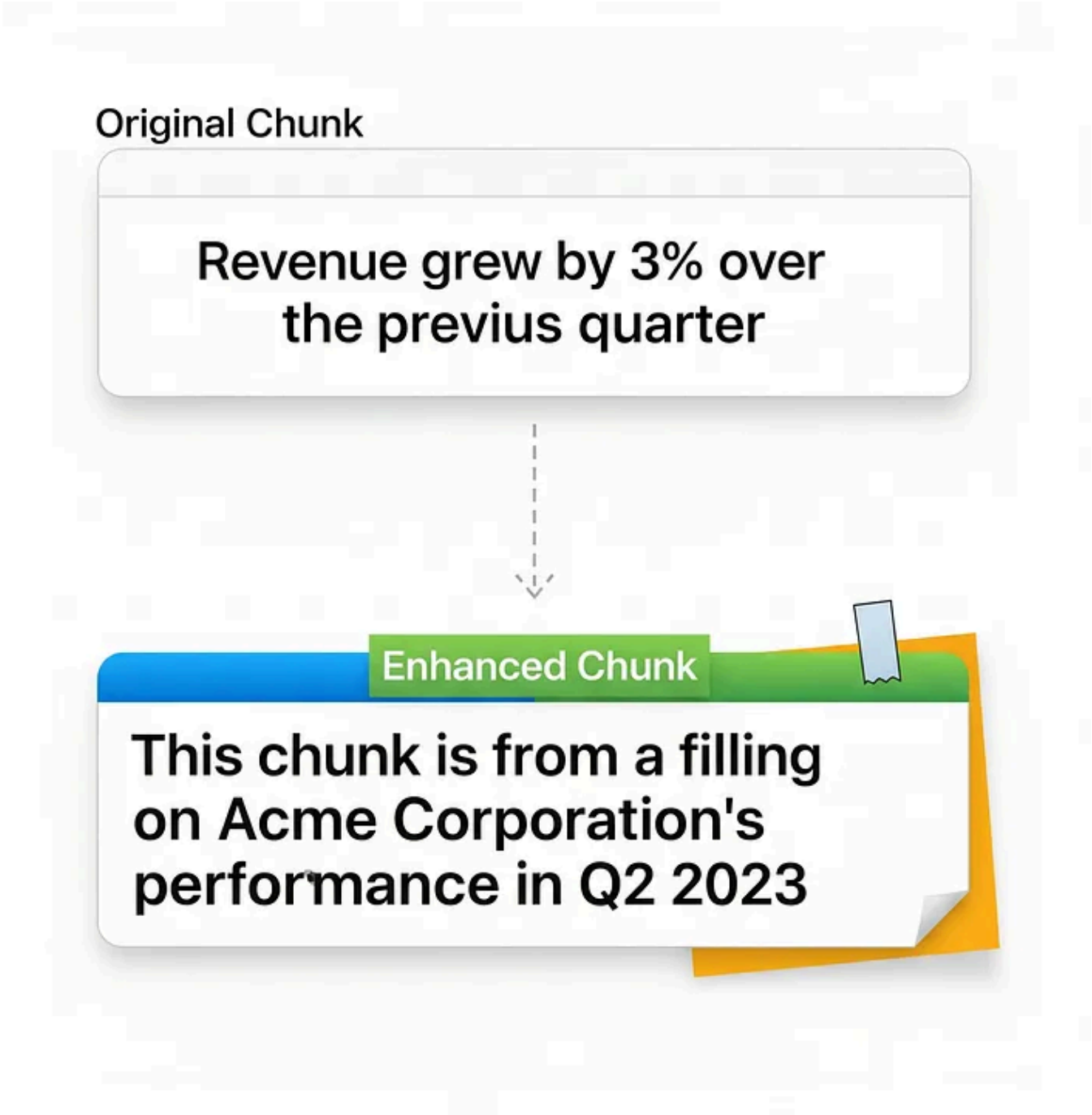
1. Take your document and chunk it normally

2. For each chunk, send both the full document and the specific chunk to an LLM
3. Use a prompt that instructs the LLM to generate a concise contextual statement
4. Prepend this context to the original chunk
5. Generate embeddings and update your BM25 index using these enriched chunks

The prompt engineering here is crucial. You're essentially asking the LLM to play detective: "Given this full document and this specific chunk, write a brief context that would help someone understand where this chunk fits in the bigger picture."

The Numbers: Does It Actually Work?

Anthropic's benchmarks are impressive. Using contextual embeddings alone, they saw a 35% reduction in retrieval failure rates (from 5.7% to 3.7% for top-20 retrieval). When they combined contextual embeddings with contextual BM25, the improvement jumped to 49% fewer failures.

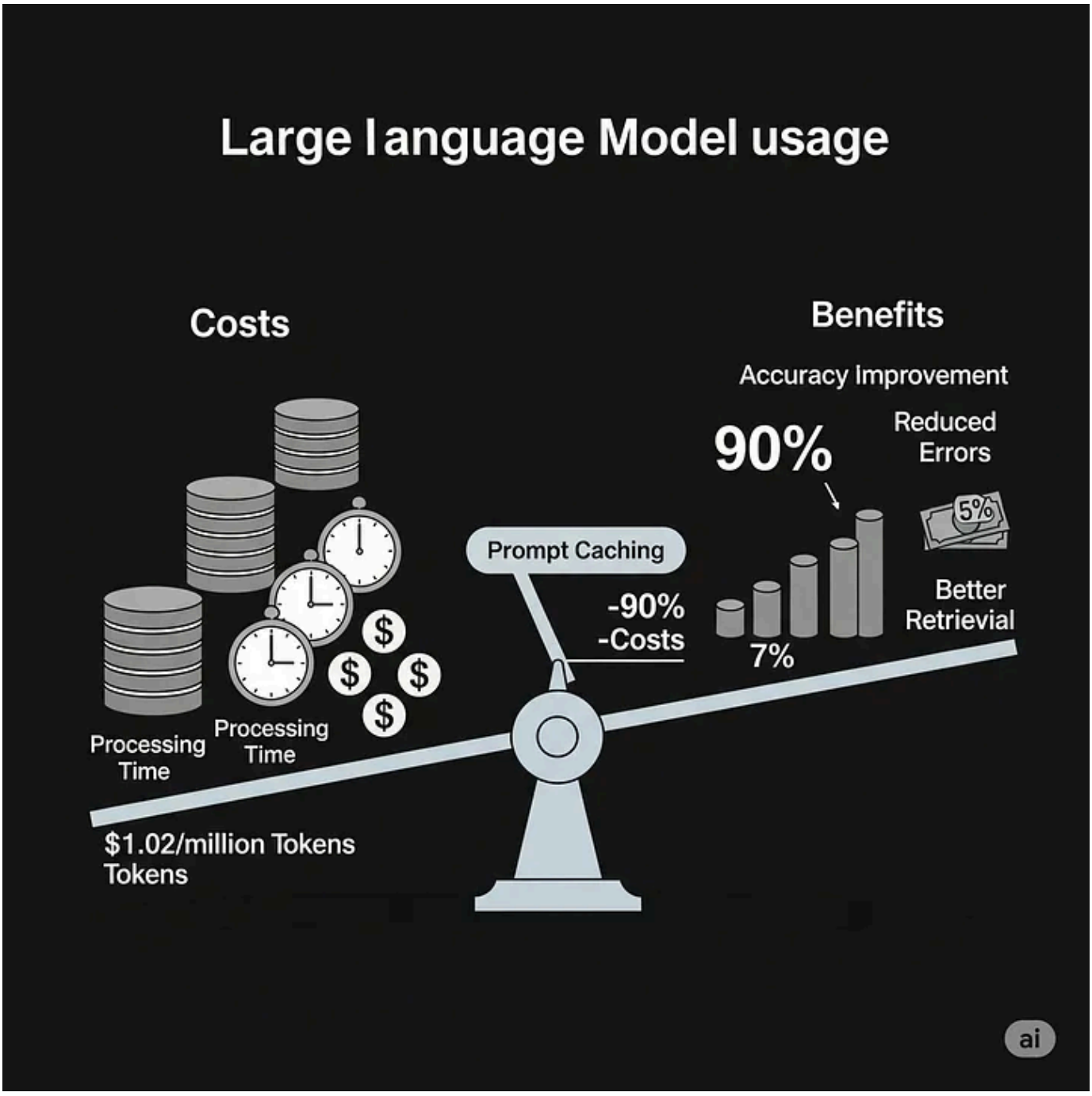


But here's where it gets really interesting. When they added a re-ranker on top of everything, the average retrieval error rate dropped from 5.7% to just 1.9%. That's a massive improvement for what amounts to a preprocessing step and some clever prompt engineering.

The Trade-offs: There's No Free Lunch

Before you rush to implement this, let's talk about costs. Adding context means:

1. **Larger chunks**: Each chunk grows by 50–100 tokens, which slightly reduces how many you can fit in your LLM's context window
2. **Processing costs**: You need to run an LLM call for every single chunk during indexing
3. **Storage overhead**: Your vector database and search indices get bigger



Anthropic estimates about $1.02 per million tokens for the one-time processing cost. That might sound small, but if you're dealing with millions of documents, it adds up. The good news is that features like prompt caching can reduce this by up to 90%, especially when processing similar documents.

When to Use Contextual Retrieval (And When to Skip It)

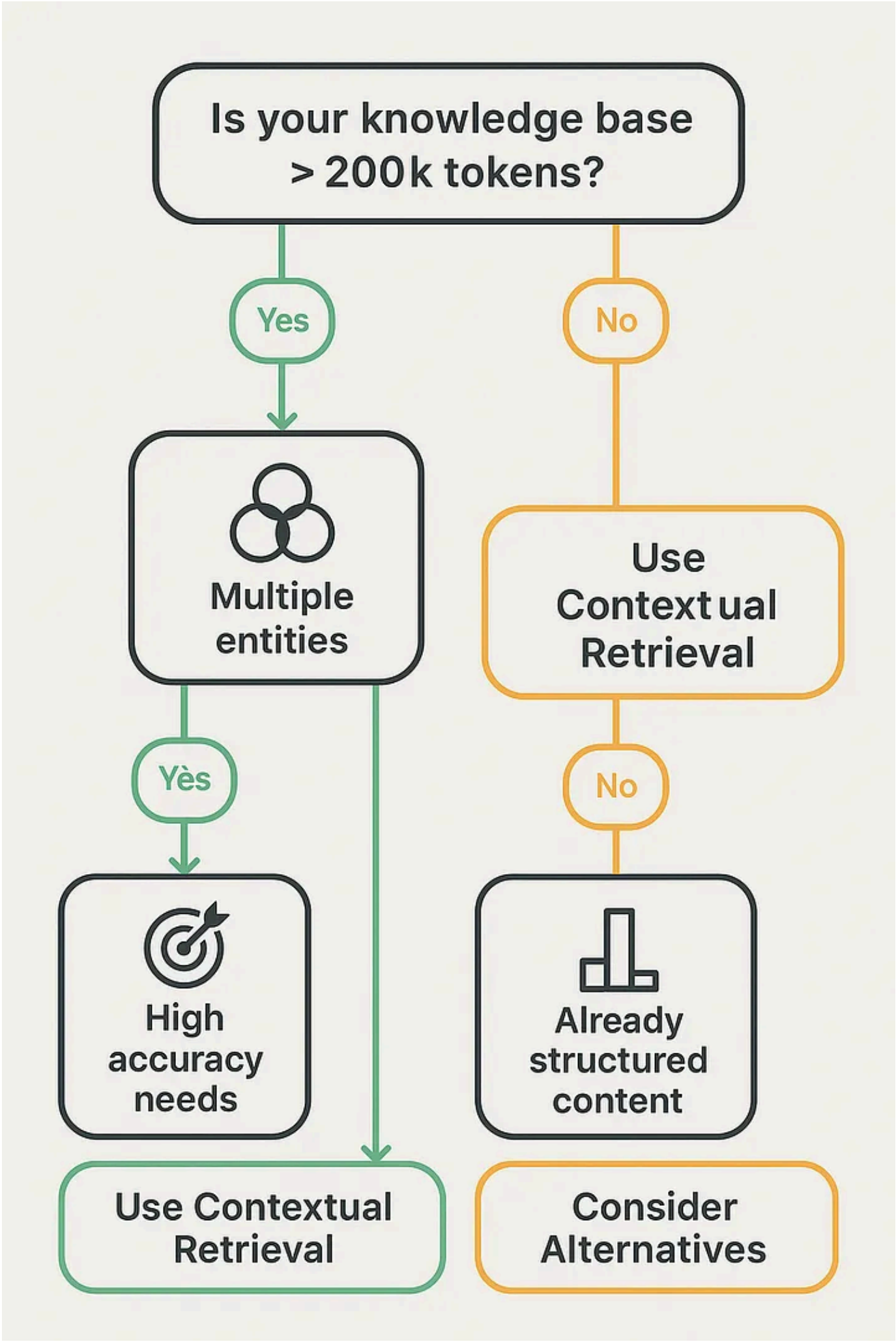
Here's a practical decision tree:

Use contextual retrieval when:

- Your documents contain multiple entities, time periods, or contexts
- You need high accuracy for specific, targeted queries
- Your knowledge base is too large for long-context LLMs (over 200K tokens)
- The cost of retrieval errors is high (think legal documents, financial reports)

Consider alternatives when:

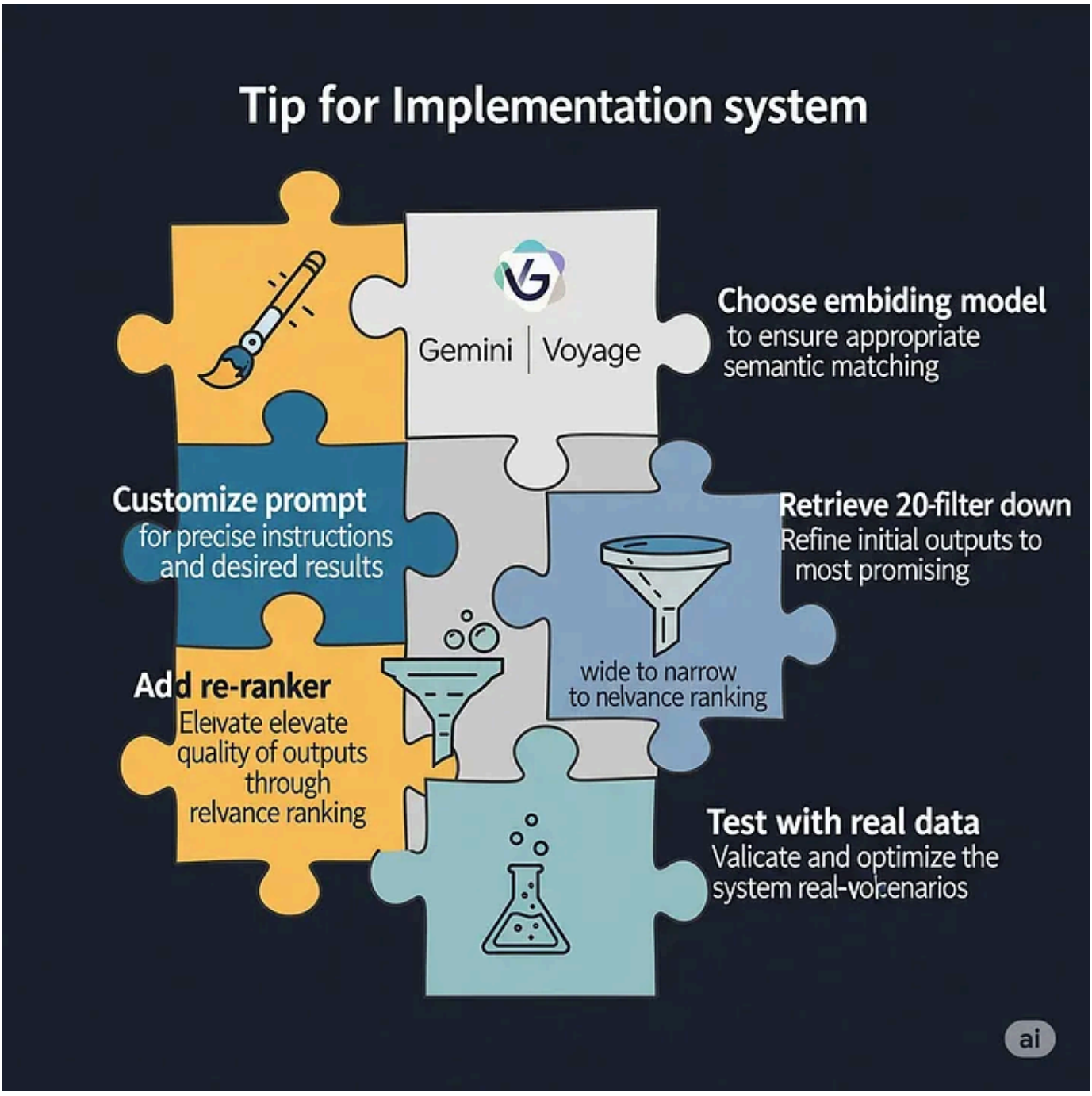
- Your knowledge base is small (under 200K tokens or 1MB if you are using Gemini for example) — just use long-context LLMs
- Your documents are already highly structured with clear context
- You're working with uniform content where chunks are self-contained


Is your knowledge base > 200k tokens?
Yes
No
Multiple entities
Use Contextual Retrieval
Yès
No
High accuracy needs
Already structured content
Use Contextual Retrieval
Consider Alternatives

Implementation Tips from the Trenches

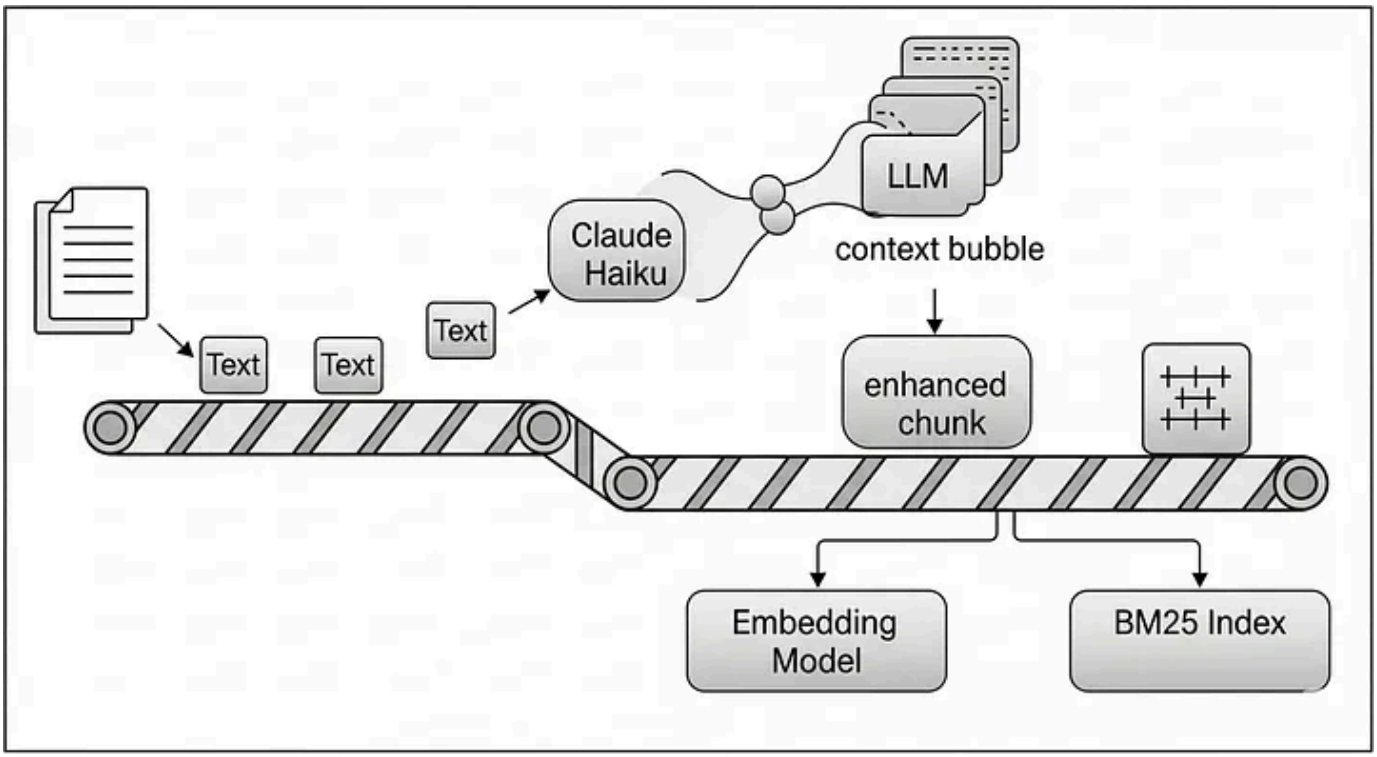
Based on Anthropic's experiments and the code they've shared, here are some practical tips:

1. **Customize your context prompt**: A prompt for financial documents needs different context than one for technical documentation. Experiment and iterate.
2. **Choose your embedding model wisely**: In their tests, Google's Gemini Text-004 performed best, with Voyage embeddings also showing strong results.
3. **Don't skimp on the re-ranker**: The combination of contextual retrieval plus re-ranking showed the best results. Consider using services like Cohere's re-ranking API.
4. **Retrieve more, then filter**: Anthropic found success retrieving 20 chunks initially, then using the re-ranker to select the best ones.
5. **Test with your actual data**: The provided benchmarks are on specific datasets. Your mileage may vary based on your document types and query patterns.



The Bigger Picture: RAG Isn't Dead

With LLMs sporting ever-larger context windows, you might wonder if RAG is becoming obsolete. Anthropic's work suggests otherwise. Even with 200K+ token context windows, RAG remains more cost-effective and scalable for large knowledge bases. Contextual retrieval makes RAG better, not obsolete.



Think of it this way: long-context LLMs are like having a photographic memory for everything you're currently looking at. RAG is like having a well-organized library with an intelligent librarian. Contextual retrieval just gave that librarian a much better card catalog.

Wrapping Up: Small Changes, Big Impact

Contextual retrieval isn't a revolutionary new architecture or a complex neural network innovation. It's a clever preprocessing step that acknowledges a simple truth: context matters. By ensuring each chunk carries its own context, we're not fighting against the limitations of embedding models or keyword search — we're working with them.

For developers building RAG systems, this technique offers a relatively straightforward way to significantly improve retrieval accuracy. Yes, there are costs and trade-offs, but for applications where accuracy matters, the investment is likely worth it.

The next time your RAG system returns a frustratingly vague answer, remember: it might just need help remembering where it put its keys. Contextual retrieval ensures each chunk comes with its own map back home.

. . .

Ready to implement contextual retrieval? Anthropic has shared their code with working examples. Start with their notebooks and adapt the approach to your specific use case. Your future self (and your users) will thank you.

. . .